Prog II:

Fundamentos do C++

Profª. Tainá Isabela

Sintaxe básica do C++

Todo programa C++ precisa de pelo menos uma função chamada main, que é o ponto de entrada da execução. A estrutura típica é:

```
#include <iostream>
using namespace std;

int main() {
    // Corpo do programa
    return 0;
}
```

Tipos de dados primitivos

A linguagem C++ é fortemente tipada, ou seja, cada variável precisa ter um tipo de dado explicitamente declarado. Isso permite ao compilador saber como armazenar, manipular e interpretar os valores na memória,.

Em C++, tipos primitivos são os tipos básicos que armazenam dados simples, como números, caracteres e valores lógicos. São a base para a construção de variáveis e manipulação de informações no programa.

Tipos de dados primitivos

TIPO	PALAVRA-CHAVE	DESCRIÇÃO	EXEMPLO DE USO
char	`char`	Armazena um caractere ASCII (1 byte)	`char letra = 'A';`
int	`int`	Armazena números inteiros (tamanho varia)	`int idade = 25;`
float	`float`	Números reais com precisão simples (4 bytes)	`float altura = 1.75f;`
double	`double`	Números reais com precisão dupla (8 bytes)	`double peso = 70.5;`
wchar_t	`wchar_t`	Caractere largo Unicode (2 ou 4 bytes)	`wchar_t simbolo = L'Ω';`
bool	`bool`	Valores lógicos: `true` ou `false`	`bool ativo = true;`

Modificadores de tipo para inteiros

C++ permite modificar o tipo int para ajustar o tamanho e o intervalo dos valores:

MODIFICADOR	DESCRIÇÃO	EXEMPLO
`short`	Inteiro curto (normalmente 2 bytes)	`short idade = 20;`
`long`	Inteiro longo (normalmente 4 ou 8 bytes)	`long populacao = 1000000;`
`unsigned`	Inteiro sem sinal (apenas valores positivos)	`unsigned int idade = 25;`

Modificadores de tipo para inteiros

C++ permite modificar o tipo int para ajustar o tamanho e o intervalo dos valores:

MODIFICADOR	DESCRIÇÃO	EXEMPLO
`short`	Inteiro curto (normalmente 2 bytes)	`short idade = 20;`
`long`	Inteiro longo (normalmente 4 ou 8 bytes)	`long populacao = 1000000;`
`unsigned`	Inteiro sem sinal (apenas valores positivos)	`unsigned int idade = 25;`

Variáveis e constantes

Variáveis são espaços nomeados na memória usados para armazenar valores que podem ser alterados durante a execução do programa. Para declarar uma variável em C++, você deve:

- Especificar o tipo de dado;
- Dar um nome (identificador);
- Opcionalmente, inicializar com um valor;

```
tipo nome_da_variavel;           // declaração simples
tipo nome_da_variavel = valor;   // declaração com inicialização
```

Variáveis e constantes

```
int idade;                // declaração
idade = 25;               // atribuição posterior

float altura = 1.75;      // declaração e inicialização

char letra = 'A';
```

Também é possível declarar várias variáveis do mesmo tipo na mesma linha:

```
int a = 10, b = 20, c = 30;
```

Variáveis e constantes

Constantes são espaços de memória cujo valor não pode ser alterado após a inicialização. Em C++, existem duas formas principais de criar constantes:

Usando o modificador const

```
const tipo nome_constante = valor;

const double PI = 3.14159;
const int DIAS_DA_SEMANA = 7;
```

- O valor deve ser atribuído na declaração e qualquer tentativa de modificar depois resultará em erro de compilação.
- Variáveis const podem ser usadas em operações como qualquer outra variável, exceto para alteração de valor.

Variáveis e constantes

Usando #define (pré-processador)

```
#define NOME_CONSTANTE valor

#define TAXA_JUROS 0.05
```

- Não usa ponto-e-vírgula ao final.
- É uma substituição textual feita antes da compilação.

Variáveis e constantes

CARACTERÍSTICA	VARIÁVEL	CONSTANTE (`CONST` OU `#DEFINE`)
Valor pode mudar?	Sim	Não
Inicialização obrigatória?	Não	Sim (para `const`)
Sintaxe	`int x = 5;`	`const int x = 5;` ou `#define X 5`

Variáveis e constantes

- Use const para valores fixos que não devem ser alterados;
- Prefira const a #define para constantes com tipo, pois oferece mais segurança e integração com o compilador;
- Dê nomes significativos e em maiúsculas para constantes;

Escopo de variáveis

Em C++, o escopo de uma variável determina onde ela pode ser acedida e utilizada dentro do programa. O escopo é o contexto em que um nome (como o de uma variável) é visível e válido para uso.

Escopo local:
Uma variável declarada dentro de uma função ou bloco (delimitado por {}) só existe e pode ser usada dentro desse bloco. Ao sair do bloco, a variável é destruída e seu valor é perdido.

```
void exemplo() {

int x = 10; // x só existe dentro desta função

}
```

Escopo de variáveis

Escopo global:

Variáveis declaradas fora de qualquer função, classe ou bloco são globais. Elas podem ser acedidas de qualquer lugar do ficheiro após a sua declaração.

```
int y = 20; // variável global

int main() {

cout << y << endl; // acesso permitido return 0;

}
```

Escopo de variáveis

Escopo de bloco:

Uma variável declarada dentro de um bloco {} (por exemplo, dentro de um laço ou condicional) só é visível dentro desse bloco.

```
if (true) {

int z = 5; cout << z << endl; // ok

}

// cout << z << endl; // erro: z não existe aqui
```

Escopo de variáveis

Escopo de classe:

Variáveis declaradas dentro de uma classe são chamadas de membros e têm escopo de classe. Elas podem ser acedidas por métodos da própria classe e, dependendo do modificador de acesso, por outras partes do programa.

Escopo de namespace:

Variáveis declaradas dentro de um namespace ficam visíveis do ponto de declaração até o final do namespace e podem ser acedidas usando o operador de resolução de escopo ::.

```
#include <iostream>
using namespace std;

int globalVar = 100; // escopo global

void funcao() {
    int localVar = 10; // escopo local
    cout << "Local: " << localVar << endl;
    cout << "Global: " << globalVar << endl;
}

int main() {
    funcao();
    // cout << localVar << endl; // erro: localVar não existe aqui
    cout << "Global no main: " << globalVar << endl;
    return 0;
}
```

Escopo de variáveis

- Variáveis com o mesmo nome podem existir em escopos diferentes sem conflito, pois cada uma é visível apenas em seu próprio contexto.
- O escopo também determina o tempo de vida da variável: variáveis locais são criadas e destruídas a cada execução do bloco, enquanto globais existem durante toda a execução do programa.
- Evite variáveis globais quando possível, prefira escopos mais restritos para evitar erros e tornar o código mais seguro e organizado.

Operadores - Aritméticos

OPERADOR	OPERAÇÃO	EXEMPLO	RESULTADO
+	Adição	a + b	Soma
–	Subtração	a - b	Diferença
*	Multiplicação	a * b	Produto
/	Divisão	a / b	Quociente
%	Módulo	a % b	Resto inteiro

Operadores - Atribuição

OPERADOR	USO	EQUIVALENTE
=	a = b	
+=	a += b	a = a + b
-=	a -= b	a = a - b
*=	a *= b	a = a * b
/=	a /= b	a = a / b
%=	a %= b	a = a % b

Operadores - Incremento e Decremento

OPERADOR	OPERAÇÃO	EXEMPLO
==	Igualdade	a == b
!=	Diferente	a != b
>	Maior que	a > b
<	Menor que	a < b
>=	Maior ou igual	a >= b
<=	Menor ou igual	a <= b

Operadores - Lógicos

OPERADOR	OPERAÇÃO	EXEMPLO
&&	E	(a > 0 && b < 5)
||	OU	(a > 0
!	NÃO	!(a == b)

Operadores - Bitwise (Bit a Bit)

OPERADOR	OPERAÇÃO	EXEMPLO
&	E bit a bit	a & b
|	OU bit a bit	a | b
^	XOR bit a bit	a ^ b
~	NOT bit a bit	~a
<<	Desloca à esq.	a << 1
>>	Desloca à dir.	a >> 1

Manipulam bits individuais dos operandos

Ordem de Precedência

- Parênteses () têm a maior prioridade.
- Depois, multiplicação/divisão/módulo (*, /, %).
- Por fim, adição/subtração (+, -).
- Operadores com mesma precedência são avaliados da esquerda para a direita.

```
#include <iostream>
using namespace std;

int main() {
    int a = 5, b = 2;
    cout << "Soma: " << a + b << endl;
    cout << "Subtração: " << a - b << endl;
    cout << "Multiplicação: " << a * b << endl;
    cout << "Divisão: " << a / b << endl;
    cout << "Resto: " << a % b << endl;
    cout << "Maior? " << (a > b) << endl;
    cout << "Igual? " << (a == b) << endl;
    cout << "E lógico? " << (a > 0 && b > 0) << endl;
    return 0;
}
```

Conversão entre tipos de dados

Conversão de tipos (type conversion) é o processo de transformar um valor de um tipo de dado para outro. Em C++, isso pode ocorrer de forma implícita (automática) ou explícita (forçada pelo programador).

Conversão Implícita

A conversão implícita ocorre automaticamente quando se atribui um valor de um tipo a uma variável de outro tipo, desde que a conversão seja segura (sem risco de perda significativa de dados).

```
int inteiro = 10;
double real = inteiro;
// int é convertido implicitamente para double
```

Conversão entre tipos de dados

Conversões implícitas também ocorrem em expressões envolvendo diferentes tipos:

```
float f = 3.5f;
int i = 2;
float resultado = f + i; // i é convertido para
float
```

Observação: Conversões implícitas de tipos maiores para menores (por exemplo, double para int) podem causar perda de dados e gerar avisos do compilador.

Conversão entre tipos de dados

Conversão Explícita (Type Casting)

Quando é necessário garantir ou forçar a conversão, utiliza o type casting. A sintaxe clássica é:

```
(tipo) valor
```

```
double x = 7.9;
int y = (int)x;
// y recebe 7, parte decimal é descartada
```

Conversão entre tipos de dados

Desde o C++ moderno, também é possível usar os operadores de casting específicos:

- static_cast<tipo>(valor);
- reinterpret_cast<tipo>(valor);
- const_cast<tipo>(valor);
- dynamic_cast<tipo>(valor);

```
float f = 9.8;
int i = static_cast<int>(f); // i recebe 9
```

Conversão entre tipos booleanos, inteiros e strings

bool para int:

```
int x = (int)true; // x = 1
```

int para bool:

```
bool b = (bool)10;
// b = true (qualquer valor diferente de zero é true)
```

Numérico para string:
Use **to_string**:

```
int n = 42;

string s = to_string(n); // "42"
```

Conversão entre tipos booleanos, inteiros e strings

String para numérico:
Use funções como **stoi, stof, atof, atoi:**

```
string s = "123";

int n = stoi(s); // n = 123
```

Conversão de ponteiros

Em C++, ponteiros podem ser convertidos, especialmente quando se trabalha com ponteiros genéricos (**void***). Para acessar o dado corretamente, é necessário fazer o cast para o tipo correto antes de usar.

```
void* ptr = ...;
int* pInt = (int*)ptr;
```

```
#include <iostream>
#include <string>
using namespace std;

int main() {
    double d = 5.7;
    int i = (int)d; // conversão explícita
    cout << "int: " << i << endl;

    string s = to_string(i); // int para string
    cout << "string: " << s << endl;

    string s2 = "123";
    int n = stoi(s2); // string para int
    cout << "int convertido: " << n << endl;

    bool b = (bool)n;
    cout << boolalpha << "bool: " << b << endl;

        return 0;
    }
```

Conversão

- Prefira conversões explícitas quando houver risco de perda de dados.
- Use funções padrão (**to_string, stoi**, etc.) para conversão entre string e tipos numéricos.
- Fique atento a avisos do compilador sobre conversões inseguras.

Saída de Dados com cout

O objeto **cout** é usado para exibir informações no monitor. Ele utiliza o operador de inserção << para enviar dados ao fluxo de saída padrão:

```
cout << "Olá, mundo!" << endl;
```

Para quebrar linha, use **endl** ou **\n**.

Entrada de Dados com cin

O objeto **cin** é utilizado para ler dados do teclado, usando o operador de extração >>:

```
int numero;
cout << "Digite um número: ";
cin >> numero;
cout << "Você digitou: " << numero << endl;
```

cin pode ser usado para ler diferentes tipos de dados (int, float, char, etc.).

Entrada de Dados com cin

Para ler múltiplos valores de uma vez:

```
int a, b;

cout << "Digite dois números: ";

cin >> a >> b;
```

Leitura de Strings com Espaços: getline

Para ler uma linha inteira (incluindo espaços), use a função **getline**:

```
string nome;
cout << "Digite seu nome completo: ";
getline(cin, nome);
cout << "Olá, " << nome << "!" << endl;
```

getline(cin, nome) lê até o Enter ser pressionado.

Resumo dos Objetos de E/S

- **cout**: saída padrão (monitor);
- **cin**: entrada padrão (teclado);
- **cerr**: saída padrão de erros (não bufferizada);
- **clog**: saída padrão de erros (bufferizada);

Namespaces

Um namespace (espaço de nomes) é uma região declarativa que fornece um escopo para identificadores (nomes de tipos, funções, variáveis, etc.) dentro dele. O principal objetivo é organizar o código em grupos lógicos e evitar conflitos de nomes, especialmente quando se utiliza várias bibliotecas ou projetos grandes.

Namespaces

```
namespace meuEspaco {
    int valor = 10;
    void mostra() {
        std::cout << "Valor: " << valor << std::endl;
    }
}
```

Namespaces

Existem três formas principais de aceder membros de um namespace:

FORMA DE ACESSO	EXEMPLO DE USO
Nome totalmente qualificado	`meuEspaco::mostra();`
Declaração using para um item	`using meuEspaco::mostra;` `mostra();`
Diretiva using para todo namespace	`using namespace meuEspaco;` `mostra();`

Namespaces

```
#include <iostream>
namespace exemplo {
    void ola() { std::cout << "Olá do namespace!" << std::endl; }
}

int main() {
    exemplo::ola(); // acesso qualificado
    using exemplo::ola;
    ola();          // acesso direto após using
    return 0;
}
```

Namespaces Aninhados

```
namespace externo {
    int x;
    namespace interno {
        int y;
    }
}
externo::x = 1;
externo::interno::y = 2;
```

Namespaces anónimos

Um namespace sem nome é chamado de anónimo. Ele restringe o acesso aos identificadores apenas ao arquivo onde foi declarado:

```
namespace {
    int segredo = 42;
}
```

Namespaces - Boas Práticas

- Prefira o uso qualificado (std::cout) em projetos grandes ou arquivos de cabeçalho.
- Use using namespace ... apenas em arquivos fonte pequenos ou funções isoladas.
- Use namespaces para organizar bibliotecas e evitar colisões de nomes.

Questionário